

Supplementary Material: Evidence Sources and Reproduction Protocols

This supplement accompanies the manuscript entitled ‘Verifiable Atomic Task Graph Relation Repair and Planning-Level Parallel Planning for Dual-Unit Collaborative Tasks’. Appendices A and B retain the evidence protocols and diagnostics; Appendix C contains the regression matrices, derived-input tests, and explanations transferred from the main text. Tables in the main text and supplementary tables are cited separately.

Appendix A. Evidence Sources, Data Construction, and Reproduction Protocols

A.1 Evidence roles and scope

The main experiment comprises 160 controlled records across four topologies, together with component, single-fault, and field-perturbation evaluations. Supplementary experiments comprise 24 plans generated directly by GPT-6, a six-setting comparison on 96 relation-information variants, 216 relation-deletion variants, 168 duration records, 24 branching-structure stress graphs, and 24 MILP solutions on the same accepted graphs. Derived variants and repeated timing runs are not counted as new model outputs or independent tasks.

The makespan, waiting time, utilization, and speedup in Sections 4.2–4.9 are computed from the input durations and abstract occupancy, rather than measured during physical execution. Section 4.10 separately reports fixed-skill schedule windows using the simulation clock and computes task completion and position errors from physical-state records. These measurements are kept distinct from the preceding symbolic times and from hardware measurements. Supplementary Fig. A1 and Table A1 report local wall-clock times in the earlier environment, including candidate construction, repair and audit where applicable, and list scheduling. Fig. 7 in the main text and Table C5 report the time for repair, joint audit, and strict list scheduling from an already loaded graph in CPython 3.11.14. Neither timing scope includes LLM/API calls, and no speed comparison is made across versions. Closure rate, resource-conflict rate, and joint acceptance rate describe only the checks performed on the supplied fields.

A.2 Data construction, candidate sources, and implementation parameters

The controlled generator cycles through four fixed topology templates: beverage preparation, box packing, bracket assembly, and tray sorting. With count=160, each template yields 40 duration-perturbation variants. Each record includes an instruction, discrete initial and goal states, node fields, typed relations, and duration estimates. Variants of a template share its structure and are used only to assess duration sensitivity and implementation stability; they are not treated as independent task topologies.

The controlled reference graphs and surrogate candidates are generated by the same deterministic ontology and generation framework. Surrogate candidates are obtained by deleting, replacing, or adding relations and fields. This shared provenance allows implementation branches and repair effects to be isolated precisely, but may favor the current ontology. It therefore does not constitute an independent gold standard or evidence of external validity.

No model was trained or fine-tuned. The main controlled configuration is count=160, generation-seed=13, duration-jitter=0.2, and structure-seed=0. The duration seed generates $u \in [-0.2, 0.2]$, with $d' = \text{round}(\max(0.5, d(1+u)), 2)$. Candidate-edge deletion and field perturbation use separately recorded structural-seed formulas and fixed probabilities. The four base reference topologies remain unchanged, whereas candidate structures vary deterministically with the record index and method-specific offset.

The historical archive also retains 8 LLM candidates from the current-session channel, covering beverage preparation, box packing, desktop organization, sorting, assembly, warehousing, service operations, and tool handover, with inputs, prompts, and raw responses. Section 4.5 uses the additional 24 GPT-6 Astra plans as the model experiment in the main text, with 4 plans in each of six task categories. The two batches have separate provenance and record identifiers and are not pooled with the 160 controlled main-experiment records.

The 24 GPT-6 plans were generated directly through a local Codex session. The model name was recorded from the GPT-6 Astra label displayed in that session. Task specifications, common generation instructions, first-pass raw candidates, freeze times, and SHA-256 hashes are archived; this generation channel did not require separate API calls. The session already contained method semantics and code context. Server-side sampling parameters enabling independent replay and repeated generations in independent sessions were not retained. Conclusions are therefore restricted to this batch and its conditions, without model rankings.

Model-generated fields and formal audit results are interpreted separately. Neither the raw nor repaired GPT-6 outputs serve as a gold standard for their own accuracy, and no self-referenced accuracy is reported for this batch. The main experiment uses controlled references, Section 4.8 uses public tool references, and Section 4.9 directly checks public source specifications; their reference objects and denominators differ. Between-task standard deviations in the model batch describe differences in content, not uncertainty from independent sampling.

A.3 Metric definitions, execution workflow, and reproduction artifacts

Node F1 uses sets of (id,action,object) triples. Full-label edge F1 uses non-order (source,target,type,state,resource) keys, with an empty value for an inapplicable state or resource. F1 is 1 when both compared sets are empty and 0 when only one is empty. State-, resource-, and execution-unit-field accuracy require exact set equality for each reference node identifier; a missing predicted node is counted as an error.

The missing-dependency rate is $|\text{Eref} \setminus \text{Epred}| / \max(1, |\text{Eref}|)$ over full-label non-order relations. Predicted statically parallel node pairs are computed using Eq. (8). Parallel-pair precision is $P = |\text{Ppred} \cap \text{Pref}| / |\text{Ppred}|$, recall is $R = |\text{Ppred} \cap \text{Pref}| / |\text{Pref}|$, and F1 is their harmonic mean. P is undefined when Ppred is empty. When Pref is nonempty and Ppred is empty, R and F1 are 0. When Pref is empty, both R and F1 are undefined regardless of whether the predicted set is empty. Undefined values are reported as ‘—’ and are not replaced by 0 in aggregation. Node- and edge-set F1 retain the conventions defined above; the conventions for these metrics are distinct.

Closure rate counts precondition obligations and goals according to Eq. (18), rather than the fraction of executable nodes. Precondition-list entries are counted individually, including separate obligations for the same state at different nodes; goals are counted as a set. A denominator of 0 gives a closure rate of 1. If the graph contains a cycle, no nodes are executed and only goals already satisfied initially are counted. The Boolean Closed gate requires acyclicity and closure of all node-precondition obligations; goal reachability is assessed by a separate gate.

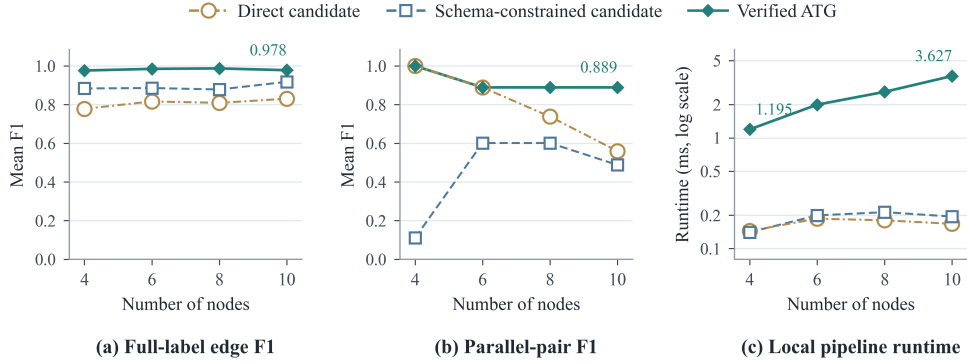
Scheduling-conflict rate is the fraction of overlapping scheduled node pairs that share a represented resource; it is 0 when no node pairs overlap. Static parallel-set Jaccard similarity is $|\text{Pref} \cap \text{Ppred}| / |\text{Pref} \cup \text{Ppred}|$ for the reference and candidate sets of statically parallel node pairs, and equals 1 when both sets are empty. Waiting time sums initial idle time and inter-task idle time on each execution-unit timeline, excluding trailing idle time. Utilization is total busy time divided by twice the makespan, and speedup is serial makespan divided by the current makespan.

The exact command for the archived main experiment is: `python -m atomic_task.cli run --count 160 --seed 13 --duration-jitter 0.2 --structure-seed 0 --out full_experiment_outputs_v7_40_internal_validity_20260907`. The pipeline exports controlled tasks, input graphs for all methods, raw candidate graphs G_0 , optimized graphs, a full-label event ledger, strictly accepted schedules, per-record tables, and summary tables.

Data and code availability: the accompanying package `ATG_reproduction_v7_70` contains frozen inputs, references, per-record results, audit and schedule records, archived MILP solutions, independent checkers, source-version identifiers, and SHA-256 manifests. The command `python reproduce_all.py results/new_run` reproduces the four-setting evaluation on identical inputs, public-specification projections, and small-scale exhaustive enumeration. Historical experiments and timing environments are documented separately, and the archived model is not called again. Third-party data retain their source, license, and original-instance attribution. The fixed-version

package is available at <https://github.com/RoboPlanner/atg-repair-planning>; protocols and downloads are indexed at <https://roboplanner.github.io/atg-repair-planning/>.

Version v7_70 removes the absolute time tolerance from precedence checks to prevent dependency overlaps involving extremely short tasks from being overlooked. Zero-length intervals, relative duration errors, and non-finite aggregate values continue to be rejected. Recalculation in v7_70 of 1,024 internal records and 3,944 archived public-tool records across four settings produced results identical to v7_69 record by record. All 58 implementation checks and 8 public-source-checker regression tests passed. Wall-clock times in historical figures and tables were measured with the archived versions and environments and do not measure runtime in v7_70.



Supplementary Fig. A1. Size trends for small controlled task graphs. (a) Full-label edge F1 and (b) parallel-pair F1, each averaged over the F1 scores of 6 instances. (c) Local pipeline runtime on a logarithmic vertical axis. Each instance is warmed up 10 times and timed 50 times; the median is taken for each instance and then averaged over 6 instances. Each size has only 1 reference topology.

A.4 Supplementary experimental protocols and the scheduling model on a fixed graph

All supplementary inputs are frozen before execution. Relation-information variants delete only the specified relations or add conservative orders between adjacent nodes in the original list, preserving all nine node-field values. For relation deletion, a fixed string containing the task identifier, deletion fraction, and seed initializes the pseudorandom generator; edges are sampled without replacement, and the actual fraction and complete deletion list are saved. Duration variants initialize random generation using the task identifier, perturbation magnitude, and seed; each original duration is multiplied by a uniform relative factor and rounded to 4 decimal places. Seeds are 11, 29, and 47. The zero-duration-perturbation setting is not counted three times.

Each layer of a branching stress graph contains w processing nodes and one join node. Each branch preserves its own state across layers, while the previous layer's

Supplementary Table A1. Controlled complexity results for 4, 6, 8, and 10 nodes.

Complexity setting	Full-label edge F1	Parallel-pair F1	Local pipeline runtime (ms)
4 nodes / direct candidate graph	0.7780±0.1895	1.0000±0.0000	0.1444±0.0212
4 nodes / schema-constrained candidate graph	0.8834±0.1258	0.1111±0.2722	0.1405±0.0292
4 nodes / Verified ATG	0.9762±0.0583	1.0000±0.0000	1.1948±0.1028
6 nodes / direct candidate graph	0.8163±0.0837	0.8889±0.0000	0.1862±0.0347
6 nodes / schema-constrained candidate graph	0.8857±0.0814	0.6012±0.0729	0.1994±0.0359
6 nodes / Verified ATG	0.9848±0.0371	0.8889±0.0000	1.9980±0.2764
8 nodes / direct candidate graph	0.8089±0.1748	0.7379±0.1827	0.1800±0.0563
8 nodes / schema-constrained candidate graph	0.8781±0.1034	0.6012±0.0729	0.2133±0.0609
8 nodes / Verified ATG	0.9872±0.0314	0.8889±0.0000	2.6089±0.3081
10 nodes / direct candidate graph	0.8305±0.1102	0.5594±0.2799	0.1673±0.0044
10 nodes / schema-constrained candidate graph	0.9177±0.0764	0.4879±0.2118	0.1946±0.0045
10 nodes / Verified ATG	0.9778±0.0344	0.8889±0.0000	3.6269±0.1537

join state supports all processing nodes in the next layer. The final join state is the goal. Widths are 2, 4, and 8, and layer counts are 2 and 4. In the shared-tool setting, nodes in the same layer compete for a capacity-1 resource; in the separate-tool setting, different branches use different resource labels. Join nodes are single or cooperative. This factorial construction gives 24 configurations and 6 actual node counts, without increasing the number of model-generated samples.

MILP variables include a nonnegative start time for each node, an upper bound on makespan, legal L/R choices for single nodes, and binary precedence choices for incomparable node pairs. Every edge in the accepted graph is fixed as a finish-to-start constraint. Binary non-overlap constraints are activated only when a task pair shares a resource or a selected base execution unit. Both unit occupancies of B are fixed to true. The big-M constant and feasible time upper bound are both set to the sum of all node durations. After solving, earliest times are recomputed from the integer assignments and ordering, followed by rechecking node coverage, state availability, durations, graph edges, resources, and L/R occupancy.

The supplementary environment uses CPython 3.11.14, NumPy 1.26.0, SciPy 1.15.3, and Windows build 26100, with processor identifier Intel64 Family 6 Model 183. All 54 tests passed, including comparison of the fixed-graph MILP with exhaustive optima on 13 small instances. All 984 supplementary pipeline evaluations replayed consistently, and all 745 accepted plans passed independent exported-plan checks; an

Supplementary Table A2. Agreement with structural and field references after controlled field perturbations.

Controlled perturbation	Full-label edge F1	State-field accuracy	Resource-field accuracy	Execution-unit-field accuracy
State-support edge deletion	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000
Resource-field omission	0.9770 $\pm$ 0.0271	1.0000 $\pm$ 0.0000	0.8250 $\pm$ 0.0145	1.0000 $\pm$ 0.0000
Precondition-state deletion	0.8698 $\pm$ 0.0686	0.8250 $\pm$ 0.0145	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000
Execution-unit label change	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	0.8250 $\pm$ 0.0145
Combined perturbations	0.8419 $\pm$ 0.0628	0.8250 $\pm$ 0.0145	0.8250 $\pm$ 0.0145	0.8250 $\pm$ 0.0145

Supplementary Table A3. Auxiliary audit and timeline metrics for the main experiment.

Method	Closure rate	Conflict rate	Critical path	Waiting time	Unit utilization
Text-order heuristic	0.6498	0.0000	13.974 $\pm$ 2.508	7.141 $\pm$ 1.685	0.636 $\pm$ 0.022
Direct candidate graph (controlled)	0.6554	0.0490	9.693 $\pm$ 2.631	2.561 $\pm$ 2.162	0.787 $\pm$ 0.074
Schema-constrained candidate graph (controlled)	0.8175	0.0437	12.253 $\pm$ 2.635	4.979 $\pm$ 2.270	0.703 $\pm$ 0.051
Reference-graph serial control	1.0000	0.0000	13.974 $\pm$ 2.508	7.141 $\pm$ 1.685	0.636 $\pm$ 0.022
Verified ATG	1.0000	0.0000	10.667 $\pm$ 2.756	2.257 $\pm$ 2.049	0.796 $\pm$ 0.075

Note: continuous timeline metrics are reported as mean $\pm$ sample standard deviation, rounded to 3 decimal places; full precision is available in the archived data. Critical path and waiting time use symbolic units. Timelines of unaccepted candidate graphs are diagnostic only.

additional 24 MILP outputs also passed. These counts refer to algorithm settings and derived records, not 984 independent tasks.

The supplementary code preserves machine time precision, rejects inapplicable relation labels before repair, and returns diagnostics for raw candidates containing cycles. Replaying the fixed 160 main-experiment inputs produces no numerical changes in the 8 summary and detailed CSV files underlying Tables 1–4. Timings in supplementary Fig. A1 and Table A1 come from their respective archived code and are not pooled with measurements from other versions.

Supplementary Table A4. Speedup of non-strict diagnostic timelines in the component ablation.

Component setting	Diagnostic speedup (160 records)
State repair disabled	1.2191 $\pm$ 0.1591
Synchronization repair disabled	1.2535 $\pm$ 0.1347
Resource orientation disabled	1.1972 $\pm$ 0.0728
Order-edge compression disabled	1.1629 $\pm$ 0.1745
Unit assignment disabled	1.0000 $\pm$ 0.0000
Full method	1.2535 $\pm$ 0.1347

Note: mean $\pm$ sample standard deviation. Non-strict timelines are not accepted outputs. Strict acceptance counts and speedup on the accepted subsets are reported in Table 4.

A.5 Reproduction protocols for repair comparisons and external archives

The perturbation configurations for the main-experiment candidates are fixed as follows. For the text-order candidate, using zero-based node index j , the last precondition is deleted when $j \bmod 3 = 1$, Cand is set to L when $j \bmod 4 = 2$, and a complete serial order is added. For the direct candidate, resources are cleared when $j \bmod 4 = 0$, the last precondition is deleted when $j \bmod 3 = 0$, and Cand is set to $\{L, R\}$ when $j \bmod 5 = 2$. For the schema-constrained candidate, only the first resource is retained when $j \bmod 4 = 0$ and the resource list has more than one entry; Cand for single nodes is broadened when $j \bmod 5 = 2$, while $\{B\}$ is retained for cooperative nodes. When there are at least 4 nodes, order relations are added between adjacent pairs starting at even indices. Each method then uses the relation-deletion probabilities listed in the main text. Method-specific seed offsets are 11, 23, and 37, respectively, and record indices start at 0. The reference-graph serial control reads the reference fields directly.

All four settings in Section 4.8 read only the same candidate and explicitly supplied initial and goal states; the reference graph is excluded from the repair interface. One-pass field matching is implemented independently, and its shared-compression variant then invokes the same order-compression module as the full method. The 160 controlled candidates and 96 GPT-6 relation views yield 1,024 evaluations, while the two external archives yield 3,944 evaluations. The current version passes 58 implementation checks and 8 public-source-checker regression tests. All 4,968 evaluations are identical to v7_69 record by record. Historical timings remain associated with their archived code versions.

The raw external files, official identifiers for the 500 test items, tool dictionary, candidate JSONL files, outputs, and SHA-256 hashes are archived, with the repository pinned to commit f1486690f4fa00030bc7035f0dcc41904889aa2c. The two models have 497 and 489 retained public outputs, respectively, and records for all four settings are saved individually. Tool names must be unique and included in the dictionary, and arguments must be lists of strings. Only complete matches to $\langle \text{node-k} \rangle$ references are parsed; self-references, out-of-range references, and references embedded in text are

rejected. Tool selection, text arguments, and task goals are not backfilled from the reference graph. Tool input/output type diagnostics use type-multiset matching and are stored separately; they are neither one of the six graph gates nor complete type validation.

External node references use sets of tool names, and external edge references use sets of directed (source tool, target tool) pairs. F1 is 1 when both sets are empty and 0 when only one is empty. Mean edge scores use all adaptable samples as the denominator, whereas acceptance counts are reported relative to all available archived outputs. Adaptation permits only single execution mode and unit symbolic duration. These results do not evaluate B occupancy, exclusive resources, or robot execution-time benefits. The public benchmark extends the external-reference evidence but does not establish an independent gold standard for robotic process semantics.

A.6 Dual-arm simulation reproduction and videos

The `simulation_expanded` package stores eight templates, 32 planning inputs, 96 planning settings, 240 physical executions, 20 preselected videos, and the per-record data and plotting code for Figs. 9–11. The environment uses CPython 3.11.14, MuJoCo 3.3.3, and NumPy 1.26.0; videos are 1280×720 at 25 frames/s. Nominal videos are fixed to the eight templates and two methods at seed 100. Four stress conditions for T08 at the same seed are also recorded as video group S12, including increased-mass and low-friction failures. Source-code hashes and parameters were frozen before the formal runs. The 16 development runs at seed −11, 9 checker regression tests, and early infrastructure failures are retained separately and excluded from the 240 runs. Regression tests include a fabricated success flag, incomplete withdrawal, grasping without lifting, insufficient dwell time, and B occupying only one unit. The 30 earlier exploratory records across three templates and the v7.70 planning package are retained separately and excluded from the frozen batch reported in the main text.

The independent checker recomputes each skill’s physical predicates. In addition to uniform samples at 40 ms intervals, exact states at all action start and end times are saved; online contact and occupancy counts at 2 ms intervals are stored separately. These discrete records cannot prove continuous-time safety. The earlier exploratory logs used contact between robot components for both `_grippers_contact`; the new batch requires both fingers of each arm to contact the object and retains separate online and sampled denominators. The `simulation_expanded` README provides reproduction commands, dependencies, licenses, raw inputs, failure logs, and file SHA-256 hashes. The package is currently distributed locally.

Appendix B. Failure Boundaries and Diagnostic Outputs

Rejection conditions include schema failure, cycles, unresolved precondition states, unreachable goals, incomplete cooperative synchronization, and unordered represented shared resources. If any such condition remains, a strict schedule is not an acceptable output.

Well-formed but incorrect fields define a separate failure boundary. Omitted resources can hide conflicts, deleted precondition states can remove obligations, and changed execution-unit fields can alter occupancy. The runtime validator does not query the controlled reference graph to recover these fields.

The diagnostic log sequentially records the stage, edge-addition/deletion operation, accept/reject decision, source, target, type, applicable state/resource label, and reason, while retaining diagnostics such as unresolved states and unreachable goals. Rejected operations are not applied to the reconstructed graph. Deleting and then rebuilding the same synchronization relation is not counted as a net repair. The original candidate graph G_0 and ordered accepted edits support replay to G_4 . Supplementary Table B1 reports diagnostic counts for each controlled setting using explicit counting units.

Supplementary Table B1. Failure diagnostics with distinct counting units; all candidates are controlled inputs.

Failure type	Counting unit	Text-order heuristic	Direct candidate graph	Schema-constrained candidate graph	Reference-graph serial control	Proposed method
Reference-inconsistent static parallelism	Node pairs	0	311	58	0	120
Missing non-order relations	Full-label edges	540	454	240	0	47
Overly conservative serialization	Order edges without state/resource justification	360	0	240	360	0
Scheduled resource conflicts	Tasks with conflicts	0	10	7	0	0
Unreachable goals	Tasks with unreachable goals	139	128	89	0	0
Reference-inconsistent occupancy fields	Nodes with inconsistent labels	120	160	120	0	120

Comparisons with additional candidate sources or solvers should record versions, parameters, and raw outputs and use an independent evaluation protocol.

Appendix C. Regression Matrices and Supplementary Comparisons

C.1 Single-fault regression and historical small-graph timing

Table C1. Nine classes of single-fault regression tests for the six-gate joint audit (n=160 per class; pass rates in %).

Controlled input	Schema-valid	Closed	Goal-reachable	Acyclic	Sync-complete	Re-source-ordered	Joint acceptance
Valid refined ATG	100	100	100	100	100	100	100
Invalid execution-unit label	0	100	100	100	100	100	0
Unresolved precondition	100	0	0	100	100	100	0
Directed cycle	100	0	0	0	100	100	0
Missing synchronization pair	100	100	100	100	0	100	0
Incorrect state-support label	0	0	0	100	50	100	0
Unsupported synchronization edge	100	100	100	100	0	100	0
Invalid resource label	0	100	100	100	100	100	0
Unordered shared resource	100	100	100	100	100	0	0
Unreachable goal	100	100	0	100	100	100	0

Table C1 injects one deterministic fault at a time into each valid refined ATG, covering invalid execution units, unresolved preconditions, cycles, missing synchronization, mislabeled state support, unsupported synchronization, invalid resources, unordered shared resources, and unreachable goals. Joint acceptance is 0 for all nine fault classes. Mislabeled state support does not trigger the synchronization gate on some non-cooperative graphs, giving a mean of 0.5000 for that gate; the schema or closure gate nevertheless causes rejection by the joint audit. This table serves as regression evidence for implementation branches.

The small-graph extension uses four sizes: 4, 6, 8, and 10 nodes. Each size has only 1 reference topology, from which each method generates 6 structurally perturbed candidate instances. Both full-label edge F1 and parallel-pair F1 are computed per instance and then averaged. The latter differs from the pooled node-pair F1 in Fig. 5.

Timing was performed on Windows 11 with CPython 3.12.14 and an Intel64 Family 6 Model 183 processor. It includes synthetic candidate construction, relation repair and audit where applicable, and list scheduling, but excludes LLM/API calls. Each

instance–method combination is warmed up 10 times and then timed 50 times using `time.perf_counter`. The median for each instance is averaged over 6 instances. Environment metadata are saved with the outputs. Background load was uncontrolled and runs were not repeated across machines; the millisecond values describe only this local execution environment.

Supplementary Fig. A1 and Table A1 report the original small-graph tests. For 4–10 nodes, the Verified ATG achieves full-label edge F1 of 0.9762–0.9872 and parallel-pair F1 of 0.8889–1.0000, while pipeline runtime increases from 1.1948 ms to 3.6269 ms. The 6-, 8-, and 10-node cases extend only the serial tail, leaving the parallel set unchanged. Each validated case has 4 true positives and 1 false positive, so parallel-pair F1 remains 0.8889. With only 1 topology per size, these results do not support claims of general size robustness, asymptotic scalability, or industrial-scale performance.

C.2 Acceptance conditions and guarantee boundaries

Table C2 lists each module's inputs, pass conditions, planning-layer guarantees, and exclusions, including the occupancy domain of the two execution units and synchronization-pairing requirements.

The guarantees in Table C2 depend on the supplied state, resource, occupancy-label, cooperative-mode, and duration fields. They do not cover facts omitted from those fields or arbitrary numbers of base execution units.

C.3 Component comparisons on paired inputs from session-generated candidates

C.4 Relation deletion and duration sensitivity

The missing-relation test deletes relations without replacement from the typed-edge sets of the 24 frozen GPT-6 candidates at nominal fractions of 10%, 30%, and 50%. Three fixed seeds, 11, 29, and 47, are used at each level, giving 216 variants. The deletion count is the nearest integer to the product of total edge count and deletion fraction, with a minimum of one. The actual fraction and edge-deletion list are saved for every variant. Node fields remain unchanged, so this test assesses whether the remaining fields support relation reconstruction, rather than semantic-field recovery.

None of the raw candidates at any of the three levels passes the joint gates. The full method restores strict acceptance to 72/72 at every level, and all 216 exported plans pass independent checks of states, precedence, resources, and execution units. Seed variants of the same base task are not treated as independent task repetitions. Because this batch has no independent reference graphs, repaired graphs are not compared with themselves to report F1 or accuracy. Actual deletion fractions differ because of integer rounding; Table C4(a) gives their ranges at each level. Net additions count relations present in the final set but absent from the input after deletion. Net deletions are 0 throughout. This measure is not equivalent to accuracy in recovering the original edge set.

The duration test fixes the original graphs and candidate units and applies uniform relative perturbations of 0, $\pm 20\%$, and $\pm 50\%$ only to positive node durations. The zero-perturbation level retains the 24 original records; each of the other two levels

Table C2. Planning-layer acceptance conditions, guarantee boundaries, and evidence mapping.

Module	Inputs and checks	Acceptance conditions and outputs	Guarantee boundary	Main evidence
Causal closure	Initial and node states; exact state support	Closed precondition obligations; reachable goals	Does not restore omitted fields	Tables 2, C1, 5, C3, and C4(a); Fig. 5
Resource ordering	Resource intersections; orientation and acyclicity checks	Declared capacity-1 resources are ordered; accepted schedules do not overlap on them	Excludes omitted resources, higher capacities, and continuous collisions	Tables 4, C1, 5, C3, and C5; Fig. 7
Dual-unit collaboration	Candidate units and modes; synchronization pairing	Exact pairing of valid state support; B occupies both L and R; legal assignment	No generalization to other unit counts or physical synchronization	Tables 4, C1, and C3; Figs. 2 and 4
Compression and scheduling	Accepted graph and durations; valid conservative order edges	Trial deletion of conservative order edges only; re-audit after deletion; strict list scheduling	No guarantee of global optimality or physical durations	Tables 3, 4, C3, and 7; Figs. 6 and 8
Rejection on failure	Six graph audits; schedule and occupancy checks	Scheduling is admitted only when all six gates pass; any failed check yields diagnostics only	Failure does not prove infeasibility; unrepresented facts are not checked	Fig. 4; Tables C1, 5, and C3; Supplementary Table B1

Note: planning-layer conditions exclude continuous trajectories, dynamics, perception errors, and hardware control.

uses three seeds to generate 72 variants, giving 168 records in total. All pass the joint audit, strict list scheduling, and exported-interval checks, showing that acceptance in this batch remains stable under the tested positive-duration changes. Differences in schedule ratios reflect changes in the task-duration structure.

C.5 Per-configuration timing of branching structures

All 12 initial shared-tool graphs are rejected because their resources are unordered, and all pass after repair. All 12 separate-tool graphs pass both before and after repair. Mean speedup is 1.0000 with shared tools and 1.4870 with separate tools, showing that additional branches do not bypass a capacity-1 bottleneck. Both single and cooperative join nodes act as inter-layer barriers and produce identical makespans in this set. Cooperative occupancy changes utilization but does not establish a general effect of the cooperative-node proportion. Median runtime for the 4 36-node configurations is

Table C3. Component comparisons with identical nodes and input relations; cells report strict acceptance counts out of 24.

Setting	All relations	Nodes only	State support only	Linear order only
Full method	24/24	24/24	24/24	24/24
State repair disabled	24/24	0/24	24/24	0/24
Synchronization repair disabled	24/24	7/24	7/24	7/24
Resource orientation disabled	24/24	2/24	2/24	24/24
Order compression disabled	24/24	24/24	24/24	24/24
Unit assignment disabled [†]	0/24	0/24	0/24	0/24

[†] With execution-unit assignment disabled, every input in this batch is rejected because its actual occupancy fails Cand/mode, not because the configuration is rejected unconditionally. For example, a single task that permits L may still pass the actual-occupancy check. This row cannot be interpreted as a comparison of scheduling algorithms. Joint acceptance for nodes-only inputs increases from 0 to 24. The full method adds a net 380 valid relations: 266 state-support, 41 synchronization, and 73 resource-order relations. The 7 inputs accepted with synchronization repair disabled contain no cooperative nodes; only 2 are accepted with resource orientation disabled. Existing obligations in the all-relations inputs are already satisfied, so disabling repair components does not reduce acceptance. Linear-order inputs can protect resources through the retained ordering and therefore remain acceptable with resource orientation disabled; its effect depends on the constraints already present in the input.

Table C4(a). Actual deletion fractions, acceptance outcomes, and net added relations under controlled relation deletion.

Nominal fraction	Actual fraction range (%)	Accepted: before → after (each out of 72)	Net added relations (total)	Consistent replay (out of 72)
10%	7.14–13.33	0→72	116	72
30%	25.00–33.33	0→72	334	72
50%	44.44–54.55	0→72	575	72

Table C4(b). Duration sensitivity with fixed GPT-6 relation graphs.

Duration perturba- tion	Strict acceptance	Estimated makespan (symbolic units)	Speedup	Paired ΔT (%)
0	24/24	17.2500 $\pm$ 4.9978	1.2411 $\pm$ 0.1402	0.00 $\pm$ 0.00
$\pm 20\%$	72/72	17.1207 $\pm$ 4.9007	1.2398 $\pm$ 0.1318	-0.72 $\pm$ 3.14
$\pm 50\%$	72/72	17.4386 $\pm$ 5.4083	1.2238 $\pm$ 0.1240	1.17 $\pm$ 6.96

Note: estimated makespan uses symbolic units. Durations and speedups are reported as mean $\pm$ sample standard deviation across records at each level. Paired ΔT first averages the makespan change relative to zero perturbation over the three seeds of the same base task, and then reports mean $\pm$ sample standard deviation across the 24 base tasks. Because the base tasks are shared, no significance inference is made, and this paired column is not interpreted as 72 independent tasks.

108.06–554.27 ms, illustrating the cost of resource orientation and repeated audits but not supporting extrapolation to large industrial graphs.

Table C5. Median post-processing time (ms) for each configuration in the branching-structure stress test.

Nodes	Width $\times$ layers	Shared/single	Shared/B	Separate/single	Separate/B
6	2 $\times$ 2	2.20	3.76	2.16	1.90
10	4 $\times$ 2	15.65	15.60	4.57	5.84
12	2 $\times$ 4	8.46	9.86	6.30	7.43
18	8 $\times$ 2	99.68	107.91	14.16	19.85
20	4 $\times$ 4	52.91	68.73	21.97	22.73
36	8 $\times$ 4	448.86	554.27	108.06	133.40

Note: each width $\times$ layer combination is crossed with four resource/join configurations, yielding 24 graphs. All 12 shared-tool initial graphs fail, whereas all 12 separate-tool initial graphs pass. After repair, all 24/24 graphs are strictly accepted. Each graph is warmed up 3 times and measured 10 times, with the median reported separately for each configuration. Timing starts from an already loaded graph and includes relation repair, joint audit, and strict list scheduling. It excludes candidate construction, file I/O, GPT-6 generation, and MILP. Its scope and environment are distinct from the historical timing in Supplementary Fig. A1. Fig. 7 shows the trends across these 24 configurations.

C.6 A resource-orientation example and fixed-skill schedule windows

In the controlled instance `sort_tray_004`, the full method orients the tray resource edge as $v_3 \rightarrow v_2$, whereas the simple combined baseline uses $v_2 \rightarrow v_3$. Their edge F1 scores are 0.9091 and 1.0000, and their speedups are 1.0000 and 1.1913, respectively. Edge F1 excludes Eorder; this counterexample reflects a difference in agreement with the reference resource orientation.

In the nominal simulation, the pick, place, and station-service windows are 6, 6, and 8 s, respectively, while approach, single-segment cooperative transfer, and withdrawal use 3, 12, and 2 s. Video groups S4–S11 each retain a pair of parallel and serial videos at seed 100. Stress-condition videos are described in A.6.